ACADEMIC CENTRAL

PRACTICE EXAMINATION

UNIVERSITY · 2ND YEAR

OBJECT-ORIENTED PROGRAMMING

PAPER 1 · PRACTICE A · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central University · 2nd year Object-Oriented Programming paper 1 · practice a.

Written by Academic Central for practice, following second-year Computer Science syllabuses at South African universities (object-oriented programming in Java). It is not a university memorandum.

How to mark: each ✓ is one mark. Accept any correct Java solution that meets the requirements, even if it differs from the model answer. Deduct at most one mark per question for small syntax slips. Accept equivalent wording in theory answers.

QUESTION 1

1.1 B ✓
1.2 B ✓
1.3 B ✓
1.4 B ✓
1.5 C ✓
1.6 A ✓
1.7 A ✓
1.8 C ✓
1.9 C ✓
1.10 B ✓

(10)

[10]

QUESTION 2

2.1 Encapsulation: data and the methods that use it are kept together, and the data is hidden behind methods ✓ (4)
Inheritance: a class reuses and extends the fields and methods of another class ✓
Polymorphism: one method call behaves differently depending on the actual object ✓
Abstraction: only the essential features are shown and the details are hidden ✓

2.2 A class is a blueprint or template that defines fields and methods; an object is one instance of a class created from it ✓. (2)
Example: Student is a class; a student called Thabo with student number 2201 is an object ✓

2.3 Overloading: same method name, different parameter lists, in the same class, decided at compile time ✓ (2)
Overriding: a subclass gives a new body to a method with the same signature as in its superclass, decided at run time ✓

2.4 Any THREE: a class can implement many interfaces but extend only one abstract class ✓; an abstract class can have instance fields, an interface only constants ✓; an abstract class can have constructors, an interface cannot ✓; abstract class methods can have any visibility, interface methods are public by default; an interface describes a capability, an abstract class a common base (3)

2.5 Is-a is inheritance, e.g. a Dog is an Animal ✓ (2)
Has-a is composition or aggregation, e.g. a Car has an Engine ✓

2.6 == checks whether the two references point to the same object ✓; equals() checks whether the objects have the same contents (if equals() is overridden) ✓ (2)

[15]

QUESTION 3

3.1 2 1 3 ✓✓✓ (count belongs to each object; total is static and shared) (3)

3.2.1 I say ... ✓ (3)
I say Woof ✓
I say Woof! ✓

3.2.2 Polymorphism (dynamic binding / method overriding) ✓: the version of sound() that runs is chosen at run time from the actual object's class, not the variable's type ✓ (2)

3.3.1 A B C ✓✓ (2)

3.3.2 Every constructor first calls its superclass constructor (super(), written or automatic), so A runs first, then B, then C ✓ (1)

3.4.1 1 10 Hi! ✓✓✓ (3)

3.4.2 Java passes arguments by value ✓: the method gets a copy of the int, but for the array it gets a copy of the reference to the same array object, so changes to the array's contents are seen by main ✓ (2)

3.5.1 B ✓ (3)
C ✓
D ✓

3.5.2 a[3] is outside the array (valid indices 0 to 2), so the exception is thrown before the println, and control jumps to the catch block ✓ (1)

[20]

QUESTION 4

4.1 Model answer: (12)

```
public class BankAccount {
    private String accountNumber;                       ✓ private fields
    private String owner;
    private double balance;

    public BankAccount(String accountNumber, String owner) {    ✓ constructor
        this.accountNumber = accountNumber;             ✓ uses this
        this.owner = owner;
        this.balance = 0;
    }
    public String getAccountNumber() { return accountNumber; }  ✓ getters
    public String getOwner() { return owner; }
    public double getBalance() { return balance; }

    public void deposit(double amount) {
        if (amount <= 0)                                 ✓ check
            throw new IllegalArgumentException("Amount must be positive");   ✓ throw
        balance += amount;                              ✓ add
    }
    public boolean withdraw(double amount) {
        if (amount > balance) return false;             ✓ not enough money
        balance -= amount;                              ✓ subtract
        return true;                                    ✓ return true
    }
    @Override
    public String toString() {                          ✓ heading
        return accountNumber + " (" + owner + "): R" + String.format("%.2f", balance);   ✓ format
    }
}
```

4.2 Model answer: (8)

```
public class SavingsAccount extends BankAccount {      ✓ extends
    private double interestRate;                        ✓ field

    public SavingsAccount(String number, String owner, double rate) {
        super(number, owner);                           ✓ super call
        this.interestRate = rate;                       ✓
    }
    public void addInterest() {
        deposit(getBalance() * interestRate);           ✓ uses getBalance()  ✓ uses deposit()
    }
    @Override
    public String toString() {
        return super.toString() + " at " + (interestRate * 100) + "%";   ✓ reuses super.toString()  ✓ adds the rate
    }
}
// Note: addInterest() on a zero balance would throw from deposit(); accept a check such as
// if (getBalance() > 0) before depositing.
```

4.3 Model answer: (6)

```
ArrayList<BankAccount> accounts = new ArrayList<>();            ✓ list
accounts.add(new BankAccount("1001", "T. Nkosi"));
accounts.add(new SavingsAccount("1002", "L. Molefe", 0.05));      ✓ both objects added
for (BankAccount acc : accounts) {                                ✓ loop
    acc.deposit(1000);                                            ✓ deposit
    if (acc instanceof SavingsAccount)                            ✓ instanceof
        ((SavingsAccount) acc).addInterest();                     ✓ cast and call
    System.out.println(acc);
}
```

4.4 Two boxes, each split into name / fields / methods ✓ (4)
BankAccount: − accountNumber : String, − owner : String, − balance : double; + BankAccount(…), + getters, + deposit(amount : double), + withdraw(amount : double) : boolean, + toString() : String ✓
SavingsAccount: − interestRate : double; + SavingsAccount(…), + addInterest(), + toString() ✓
Hollow triangle arrow from SavingsAccount to BankAccount (inheritance) ✓

[30]

QUESTION 5

5.1 Model answer: (5)

```
public interface Shape {                              ✓ interface
    double area();
    double perimeter();
}
public class Circle implements Shape {                ✓ implements
    private double radius;
    public Circle(double radius) { this.radius = radius; }   ✓ constructor
    public double area() { return Math.PI * radius * radius; }     ✓
    public double perimeter() { return 2 * Math.PI * radius; }     ✓
}
```

5.2 Model answer: (4)

```
@Override
public int compareTo(Student other) {                 ✓ heading with Student parameter
    return other.mark - this.mark;                    ✓✓ other first, so highest comes first
}                                                     ✓ returns an int
// Also accept: return Integer.compare(other.mark, this.mark);
```

5.3 Any ONE: type errors are found at compile time instead of at run time ✓; no casting is needed when taking items out ✓; the code is clearer and reusable for any type (2)

5.4 (a) HashSet (or TreeSet) ✓, because a set does not allow duplicates ✓ (4)
(b) HashMap ✓, because it finds a value (the name) by its key (the student number) in about O(1) time ✓

[15]

QUESTION 6

6.1 Checked exceptions must be caught or declared with throws, and the compiler checks this ✓, e.g. IOException ✓ (3)
Unchecked exceptions (RuntimeException and its subclasses) need not be declared, e.g. NullPointerException ✓

6.2 Model answer: (3)

```
public class InsufficientFundsException extends Exception {   ✓ extends Exception (checked)
    public InsufficientFundsException(String message) {        ✓ constructor
        super(message);                                        ✓ passes the message on
    }
}
```

6.3 Single Responsibility Principle: a class should have only one job (one reason to change) ✓✓ (4)
Low coupling: classes depend on each other as little as possible ✓
High cohesion: everything inside a class is closely related to its one purpose ✓

[10]

TOTAL: 100

END OF MARKING GUIDELINE